



Online Python Compiler and Visualizer

[Learning Python? Check out our complete Python roadmap](#)

Python3



Run

Visualize Code

```
1 arr = [10, 25, 30, 45, 50]
2 key = 30
3
4 found = False
5
6 for i in range(len(arr)):
7     if arr[i] == key:
8         print("Key found at Index", i)
9         found = True
10        break
11
12 if not found:
13     print("Key not found")
14
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status: Successfully executed

Time:
0.0000 secs

Memory:
8.988 Mb

Your Output

Key found at index 2



Online Python Compiler and Visualizer

Learning Python? Check out our complete Python roadmap [↗](#)

Python3



Run

Visualize Code

```
1 def binary_search(arr, low, high, key):
2     if low <= high:
3         mid = (low + high) // 2
4
5         if arr[mid] == key:
6             return mid
7
8         elif arr[mid] < key:
9             return binary_search(arr, mid + 1, high, key)
10
11        else:
12            return binary_search(arr, low, mid - 1, key)
13
14    return -1
15
16
17 arr = [5, 10, 15, 20, 25]
18 key = 20
19
20 index = binary_search(arr, 0, len(arr) - 1, key)
21
22 if index != -1:
23     print("Key found at index", index)
24 else:
25     print("Key not found")
26
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status: Successfully executed

Time:

0.0100 secs

Memory:

9.044 Mb

Your Output

Key found at index 3



Online Python Compiler and Visualizer

Learning Python? Check out our complete Python roadmap [↗](#)

Python3



Run

Visualize Code

```
1
2 def factorial_iterative(n):
3     fact = 1
4
5     for i in range(1, n + 1):
6         fact = fact * i
7
8     return fact
9
10
11
12 def factorial_recursive(n):
13     if n == 0 or n == 1:
14         return 1
15     else:
16         return n * factorial_recursive(n - 1)
17
18
19 n = 5
20
21 result_iterative = factorial_iterative(n)
22 result_recursive = factorial_recursive(n)
23
24 print("Factorial using Iterative Method:", result_iterative)
25 print("Factorial using Recursive Method:", result_recursive)
26
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:

0.0100 secs

Memory:

8.988 Mb

Your Output

Factorial using Iterative Method: 120
Factorial using Recursive Method: 120

Python3



Run

Visualize Code

```
1 n = 6
2 a, b = 0, 1
3
4 print("Iterative:", end=" ")
5 for i in range(n):
6     print(a, end=" ")
7     a, b = b, a + b
8 def fib(n):
9     if n <= 1:
10         return n
11     return fib(n - 1) + fib(n - 2)
12
13 print("\nRecursive:", end=" ")
14 for i in range(n):
15     print(fib(i), end=" ")
16
```

Enter Input here

If your code takes Input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	9.032 Mb

Your Output

Iterative: 0 1 1 2 3 5
Recursive: 0 1 1 2 3 5



Online Python Compiler and Visualizer

Learning Python? Check out our complete Python roadmap

Python3



Run

Visualize Code

```
1 # Matrix Multiplication
2
3 A = [[1, 2],
4       [3, 4]]
5
6 B = [[5, 6],
7       [7, 8]]
8
9 C = [[0, 0],
10      [0, 0]]
11
12 for i in range(2):
13     for j in range(2):
14         for k in range(2):
15             C[i][j] += A[i][k] * B[k][j]
16
17 print("Result:", C)
18
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:

0.0200 secs

Memory:

9.008 Mb

Your Output

Result: `[[19, 22], [43, 50]]`



Online Python Compiler and Visualizer

Learning Python? Check out our complete Python roadmap

Python3



Run



Visualize
Code

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status: Successfully executed

Time:
0.0000 secs

Memory:
8.884 Mb

Your Output

Sorted array: [3, 9, 10, 27, 38, 43, 82]

```
1 # Merge Sort
2
3 def merge_sort(arr):
4     if len(arr) <= 1:
5         return arr
6
7     mid = len(arr) // 2
8     left = merge_sort(arr[:mid])
9     right = merge_sort(arr[mid:])
10
11     result = []
12     i = j = 0
13
14     while i < len(left) and j < len(right):
15         if left[i] < right[j]:
16             result.append(left[i])
17             i += 1
18         else:
19             result.append(right[j])
20             j += 1
21
22     result += left[i:]
23     result += right[j:]
24
25     return result
26
27
28 arr = [38, 27, 43, 3, 9, 82, 10]
29
30 print("Sorted array:", merge_sort(arr))
```



Online Python Compiler and Visualizer

Learning Python? Check out our complete Python roadmap

Python3



Run



Visualize
Code

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.952 Mb

Your Output

Sorted array: [1, 5, 7, 8, 9, 10]

```
1 # Quick Sort
2
3 def quick_sort(arr):
4     if len(arr) <= 1:
5         return arr
6
7     pivot = arr[-1]
8     left = [x for x in arr[:-1] if x <= pivot]
9     right = [x for x in arr[:-1] if x > pivot]
10
11     return quick_sort(left) + [pivot] + quick_sort(right)
12
13
14 arr = [10, 7, 8, 9, 1, 5]
15
16 print("Sorted array:", quick_sort(arr))
17
```

Python3



Run



Visualize
Code

```
1 # Tower of Hanoi
2
3 def hanoi(n, source, auxiliary, destination):
4     if n == 1:
5         print("Move disk 1 from", source, "to", destination)
6         return
7
8     hanoi(n - 1, source, destination, auxiliary)
9     print("Move disk", n, "from", source, "to", destination)
10    hanoi(n - 1, auxiliary, source, destination)
11
12
13 n = 3
14 hanoi(n, 'A', 'B', 'C')
15
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	8.924 Mb

Your Output

```
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```



Online Python Compiler and Visualizer

[Learning Python? Check out our complete Python roadmap](#)

Python3



Run



Visualize
code

```
1 # GCD using Euclidean Algorithm
2
3 def gcd(a, b):
4     if b == 0:
5         return a
6     return gcd(b, a % b)
7
8
9 a = 48
10 b = 18
11
12 print("GCD =", gcd(a, b))
13
```

Enter Input here

if your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.828 Mb

Your Output

GCD = 6



Online Python Compiler and Visualizer

[Learning Python? Check out our complete Python roadmap](#)

Python3



Run



Visualize
Code

```
1 # Insertion Sort
2
3 arr = [12, 11, 13, 5, 6]
4
5 for i in range(1, len(arr)):
6     key = arr[i]
7     j = i - 1
8
9     while j >= 0 and arr[j] > key:
10         arr[j + 1] = arr[j]
11         j -= 1
12
13     arr[j + 1] = key
14
15 print("Sorted array:", arr)
16
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:

0.0100 secs

Memory:

9.016 Mb

Your Output

Sorted array: [5, 6, 11, 12, 13]



Online Python Compiler and Visualizer

[Learning Python? Check out our complete Python roadmap](#)

Python3



Run



Visualize
Code

```
1 def heapify(a, n, i):
2     m = i
3     l, r = 2*i+1, 2*i+2
4
5     if l < n and a[l] > a[m]: m = l
6     if r < n and a[r] > a[m]: m = r
7
8     if m != i:
9         a[i], a[m] = a[m], a[i]
10        heapify(a, n, m)
11
12 def heap_sort(a):
13     n = len(a)
14
15     for i in range(n//2-1, -1, -1):
16         heapify(a, n, i)
17
18     for i in range(n-1, 0, -1):
19         a[0], a[i] = a[i], a[0]
20         heapify(a, i, 0)
21
22 a = [4, 10, 3, 5, 1]
23 heap_sort(a)
24 print(a)
25
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:

0.0200 secs

Memory:

9.072 Mb

Your Output

[1, 3, 4, 5, 10]



Online Python Compiler and Visualizer

Learning Python? Check out our complete Python roadmap [↗](#)

Python3 ▾



Run



Visualize
Code

```
1 def merge_count(a):
2     if len(a) <= 1:
3         return a, 0
4
5     m = len(a) // 2
6     left, x = merge_count(a[:m])
7     right, y = merge_count(a[m:])
8
9     i = j = inv = 0
10    res = []
11
12    while i < len(left) and j < len(right):
13        if left[i] <= right[j]:
14            res.append(left[i])
15            i += 1
16        else:
17            res.append(right[j])
18            inv += len(left) - i
19            j += 1
20
21    return res + left[i:] + right[j:], x + y + inv
22
23 a = [2, 4, 1, 3, 5]
24 print("Inversions:", merge_count(a)[1])
25
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:

0.0000 secs

Memory:

8.888 Mb

Your Output

Inversions: 3



Online Python Compiler and Visualizer

[Learning Python? Check out our complete Python roadmap](#)

Python3



Run

Visualize
Code

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:

0.0100 secs

Memory:

9.124 Mb

Your Output

Maximum Subarray Sum: 7

```
1 a = [-2, -3, 4, -1, -2, 1, 5, -3]
2
3 max_sum = a[0]
4 current = a[0]
5
6 for i in range(1, len(a)):
7     current = max(a[i], current + a[i])
8     max_sum = max(max_sum, current)
9
10 print("Maximum Subarray Sum:", max_sum)
11
```



Courses Practice Compete Compiler

Upgrade to Pro

Login

Sign Up

Online Python Compiler and Visualizer

Learning Python? Check out our complete Python roadmap

Python3



Run



Visualize
Code

```
1 # Exponentiation
2
3 def iterative_power(x, n):
4     result = 1
5     for i in range(n):
6         result *= x
7     return result
8
9 def fast_power(x, n):
10     if n == 0:
11         return 1
12     half = fast_power(x, n // 2)
13     if n % 2 == 0:
14         return half * half
15     return x * half * half
16
17 x, n = 2, 10
18
19 print("Iterative:", iterative_power(x, n))
20 print("Recursive:", fast_power(x, n))
21
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status: Successfully executed

Time:

0.0100 secs

Memory:

9.184 Mb

Your Output

Iterative: 1024

Recursive: 1024



Online Python Compiler and Visualizer

Learning Python? Check out our complete Python roadmap

Python3



Run



Visualize
Code

```
1 import math
2
3 points = [(1, 2), (4, 5), (7, 8), (3, 1)]
4
5 min_dist = float('inf')
6 pair = None
7
8 for i in range(len(points)):
9     for j in range(i + 1, len(points)):
10         d = math.dist(points[i], points[j])
11
12         if d < min_dist:
13             min_dist = d
14             pair = (points[i], points[j])
15
16 print("Closest pair:", pair)
17 print("Distance =", round(min_dist, 2))
18
```

Enter Input here

if your code takes input, add it in the above box before running.

Output

Status: Successfully executed

Time:
0.0100 secs

Memory:
9.312 Mb

Your Output

Closest pair: ((1, 2), (3, 1))
Distance = 2.24